ATTENDANCE CHECKER

Statistical Code Review Report

Introduction

This report showcases results gathered from running SonarQube and CheckStyle code quality and security analysers on the project code. These results were gathered before cleaning and refactoring any of the code meaning that the statistics are quite concerning in most of the section. Code cleaning and refactoring is done based on the results and recommendations received from the SonarQube analysis which are showcased in this report.

Analysis Results

This chapter covers the main findings from the SonarQube and Checkstyle analysis.

SonarQube

SonarQube analysis overview shows the statistics of the project code. Image 1 shows the overview for this project. From the results we can see that there are 6 issues in the reliability category that need to be fixed. The code also has a pretty high duplication score that needs to be lowered. The Security Hotspots are related to debugging features that need to be disabled, which is an easy task. The code coverage score is low since the existing unit tests only cover DAO classes. Controller classes are difficult to test as they need a lot of mock data and connection to function. Unfortunately we as a team do not have the competence, or time, at the moment to add tests for these classes to raise the coverage percentage.

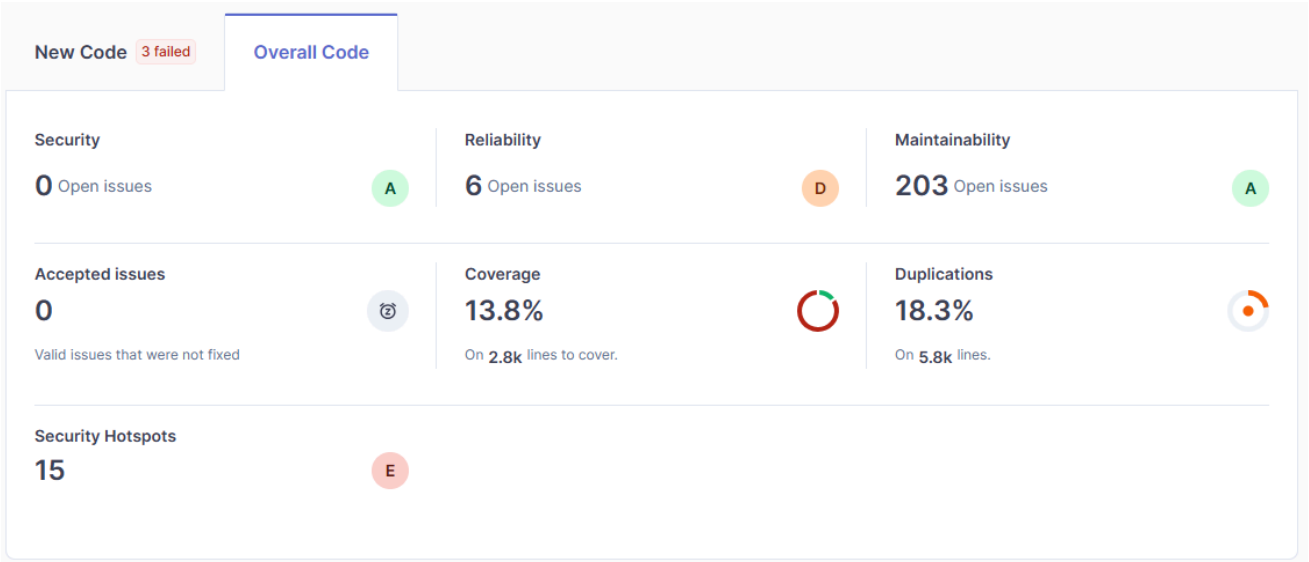

Image 1. The SonarQube analysis overview for the Attendance Checker project code.

Summary Metrics

Along with the statistics shown in the overview, the code was also analysed from other points of view. In this report we analysed the code with the three following measurements: Cyclomatic Complexity, Lines of code per method and Duplications.

Cyclomatic Complexity

Cyclomatic Complexity measures how complicated the code is. The rating starts from 1 and every decision point, i.e. if and for statements, adds a point. The higher the score, the more complex the code is. Complex code adds risks. The Cyclomatic Complexity ratings for the classes in the project range from 2 to 37. Rating below 10 is low risk, 11 to 20 is moderate risk and 21 to 50 is high risk. Some of the entity classes seem to have higher ratings even though they do not contain complex logic. These classes contain many getter and setter methods which impact on the complexity rating. As expected, some of the controller and view classes also have fairly high ratings. Image 2 shows the Cyclomatic Complexity ratings for the controller classes.

OTP1 > src > main/java > controller View as Tree Select files Navigate 9 items

Cyclomatic Complexity 132

AddStudentsController.java	15
AllCoursesController.java	14
CourseAttendanceReportController.java	27
CreateCourseController.java	2
LoginController.java	9
SelectedAttendanceCheckController.java	14
SelectedCourseController.java	16
SelectedCourseStudentsController.java	7
StudentAttendanceReportController.java	28

9 of 9 shown

Image 2. The Cyclomatic Complexity ratings for controller classes.

Lines Of Code Per Method

The project has 4160 lines of code and 298 functions. With these numbers we get an average of around 14 lines of code per method or function. There are no official guidelines that determine how many lines of code a function should have in maximum. Based on a quick Google search 14 lines seems to be pretty good. Of course the project has a lot of methods that are way longer than that and it also has many getter and setter methods that are usually one or two lines long. So you cannot ,based on this metric alone, decide that all methods are at great length.

Duplicate code

The project has 1066 duplicate lines. In percentage the duplication density is 18,8%. This is quite high and needs to be fixed with code refactoring. View and entity classes seem to have the most duplication. In view classes the high percentage is caused by the reused view elements, e.g. the side panel, that are written again in each view class. In entity classes the high scores are caused by the Teacher and Student classes that have a really similar structure. Image 3 shows the duplicate lines and percentages in all the packages of the project code.

OTP1 > src > main/java View as Tree Select files Navigate 9 items

Duplicated Lines (%) 18.8%

	Duplicated Lines (%)	Duplicated Lines
controller	4.5%	54
dao	0.0%	0
datasource	0.0%	0
entity	29.1%	240
i18n	0.0%	0
seed_data	0.0%	0
utils	0.0%	0
view	29.6%	772
Main.java	0.0%	0

Image 3. Duplicate lines per each package.

CheckStyle

CheckStyle measures whether code adheres to the certain coding standards and good practices. They are not necessary errors or bugs, more like visual, structural and readability issues.

The project code has recurring issues related to documentation, naming conventions, line lengths, magic numbers and imports. Some of these are explained in more detail in the next chapter. Image 4 shows some of the issues found with CheckStyle.

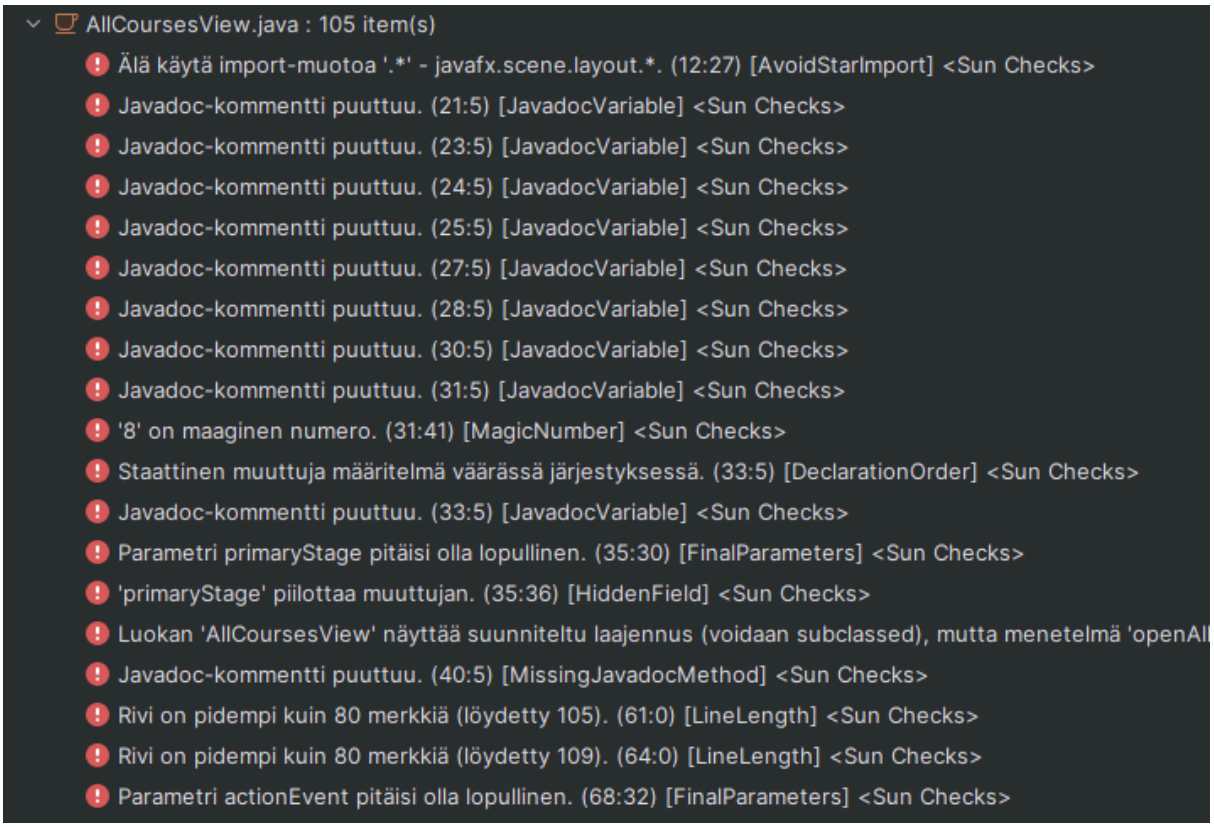

Image 4. Some CheckStyle analysis issues found in AllCoursesView class.

Issues

This section showcases all the issues found from the analysis that need to be fixed. The issues are split into two categories: High Priority and Common. Each issue is explained and it is given a solution. Fixing issues improves code quality.

High Priority Issues

High Priority issues are issues that can directly affect the functioning of the program. They might decrease the security and reliability of the program. As such, they need to be fixed as soon as possible. Below is displayed high priority issues.

SonarQube: "NullPointerException" will be thrown when invoking method”

> **Explanation:** Method can throw a NullPointerException since it is using an object that is not confirmed to be not null.
>
> **Solution:** Refactor the code to prevent the execution of a method if the used object is null.

SonarQube: “Use try-with-resources or close this "BufferedWriter" in a "finally" clause”

> **Explanation:** BufferedWriter output stream is used without closing it properly which can lead to resource leak.
>
> **Solution:** Close the stream after writing by changing the try-catch to try-with-resources or by adding a “finally” clause.

Common Issues

Common issues are ones that mostly affect the style and readability of code. They are not bugs and do not affect the functionality of the program. They should be fixed because they can affect code maintainability. Below is displayed some of the common issues found from the code.

SonarQube: Code duplication

> **Explanation:** Classes have duplicate lines and methods written in exactly the same way. This can make maintenance very difficult.
>
> **Solution:** Remove and merge duplicate methods. Call methods from their own class.

CheckStyle: Javadoc comment missing

> **Explanation:** Classes and methods are missing Javadoc comments explaining the usage of the class or method.
>
> **Solution:** Add and update Javadoc comments to all the classes and methods.

CheckStyle: Attribute should be final

Explanation: Some class attributes are not marked as final even though they are not changed later in the program.

Solution: Add a “final” statement in front of the attributes that are not changed after initialization.

CheckStyle: Class seem extendable but does not have comment explaining how to expand safely

Explanation: Code does not specify whether a class is extendable or not.

Solution: Make class final or add a Javadoc comment explaining how class is used.

CheckStyle: Local variable shadows attribute (HiddenField)

Explanation: Local variable or parameter has the same name as a class attribute. This can cause confusion.

Solution: Rename local variables and attributes so that they are not confused with class attributes.

CheckStyle: Line is longer than 80 characters

Explanation: The code line has more than 80 characters making it hard to read.

Solution: Split the code to multiple lines.

Conclusion

The SonarQube and CheckStyle analysis helped find and identify the main issues in the code. The code will be cleaned and refactored based on the findings. The goal is to fix the High Priority issues and the common issues showcased in this report.